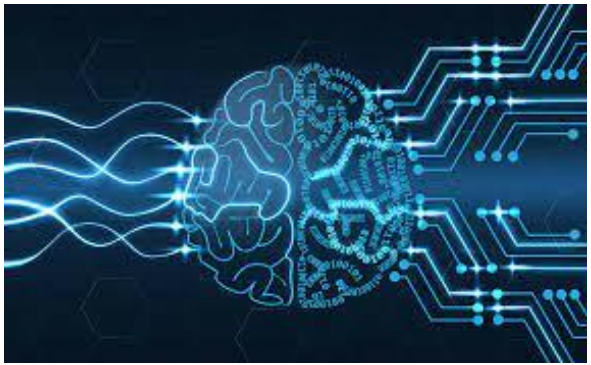
QLS612 - Advanced LLMs: Harnessing Agentic Workflows

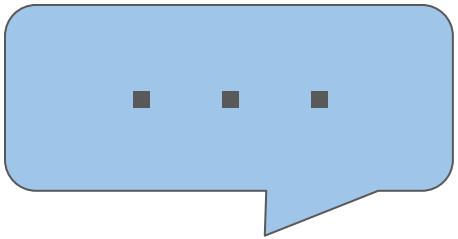
2026-05-25

Brent McPherson, PhD

Topics

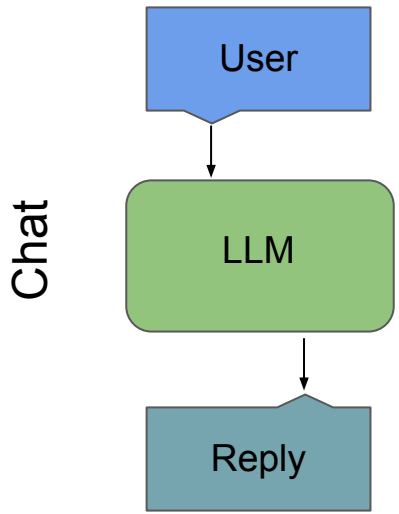
- How did we get here?
  - Chat -> Harness
- “Harnessing” AI Agents for Data Science
- Core Configuration
- Example Workflows
- Testimonials





https://www.forbes.com/sites/garydrenik/2023/01/11/large-language-models-will-define-artificial-intelligence/

How did we get here?

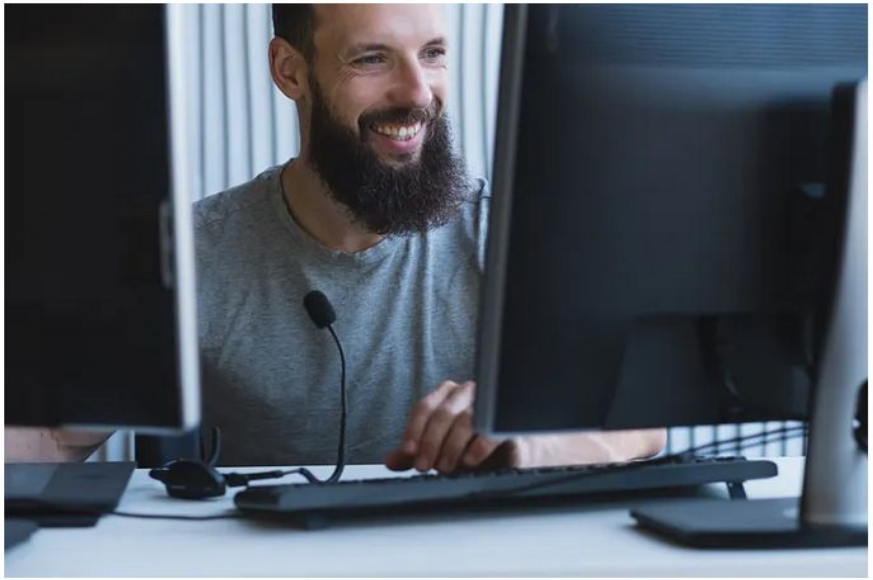

Chat
User
LLM
Reply

Why Prompt Engineering Is the Job of the Future

Why you need to be able to talk to AI software.

Kristina God Follow 4 min read · Apr 7, 2023

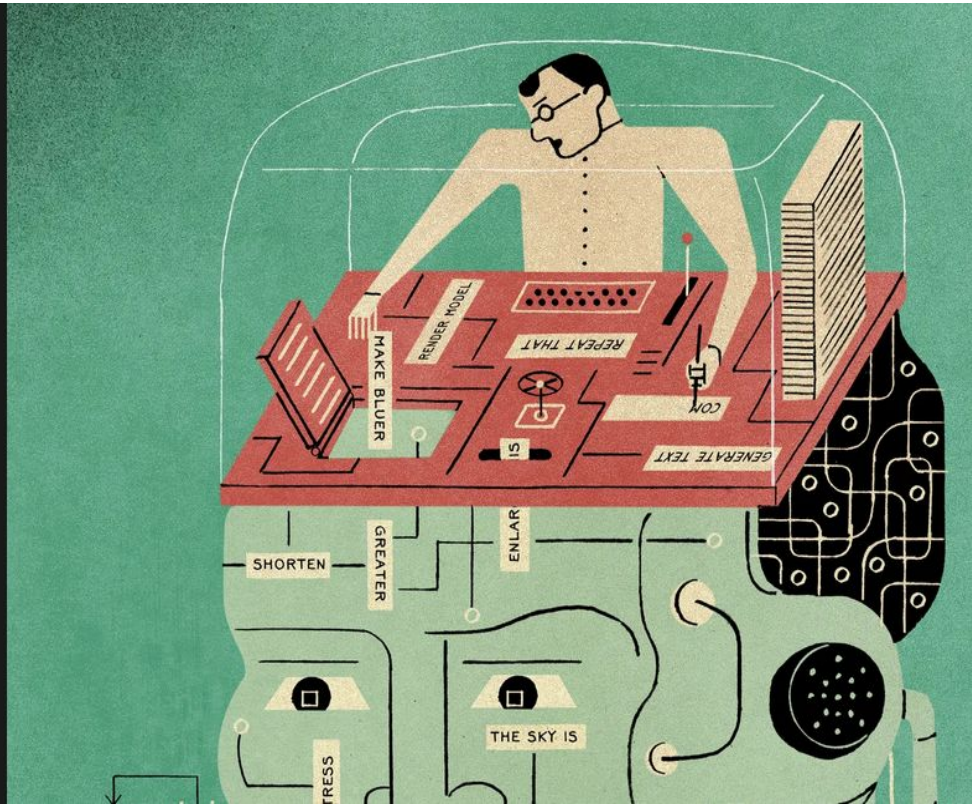
527 6



FEATURE | AI

AI PROMPT ENGINEERING IS DEAD

Long live AI prompt engineering



https://towardsdatascience.com/what-i-learned-pushing-prompt-engineering-to-the-limit-c40f0740641f; https://spectrum.ieee.org/prompt-engineering-is-dead

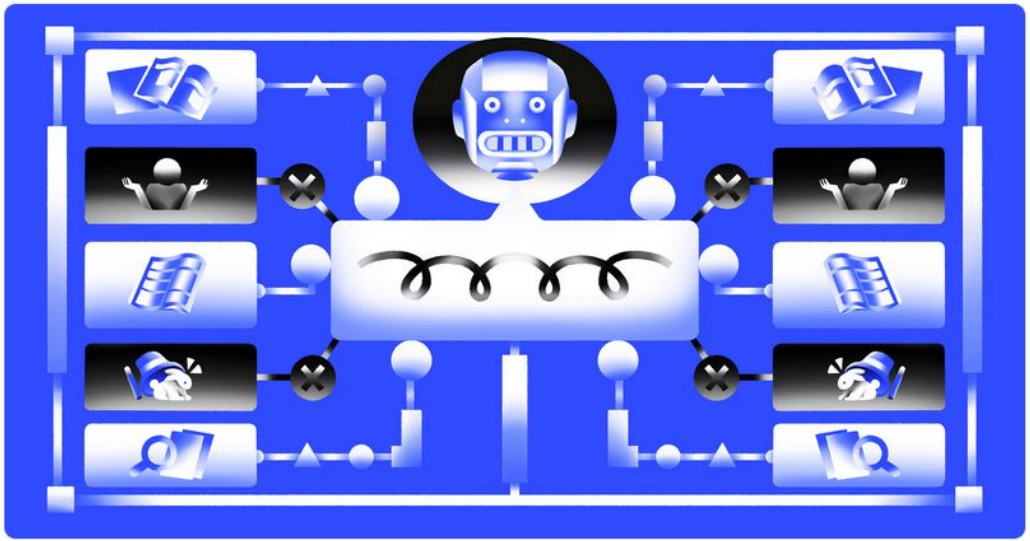
So why did it “die”?

- Good prompts need context.
- People began looking for programmatic ways to add context to prompts.
- They needed rapid access to structured chunks of information.

OCTOBER 18, 2023

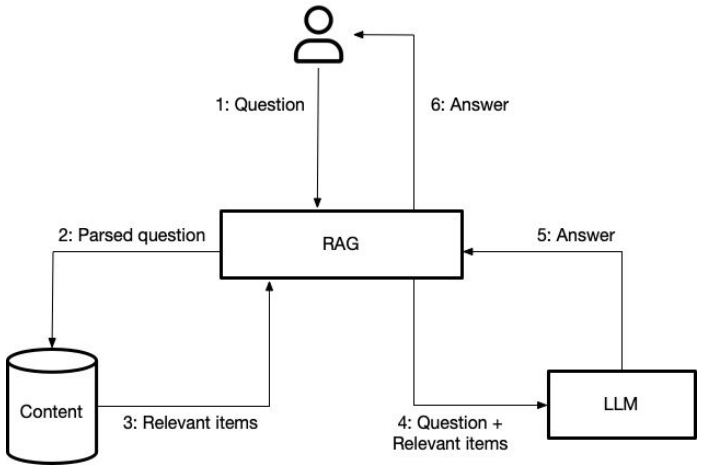
Retrieval augmented generation: Keeping LLMs relevant and current

Retrieval augmented generation (RAG) is a strategy that helps address both LLM hallucinations and out-of-date training data.

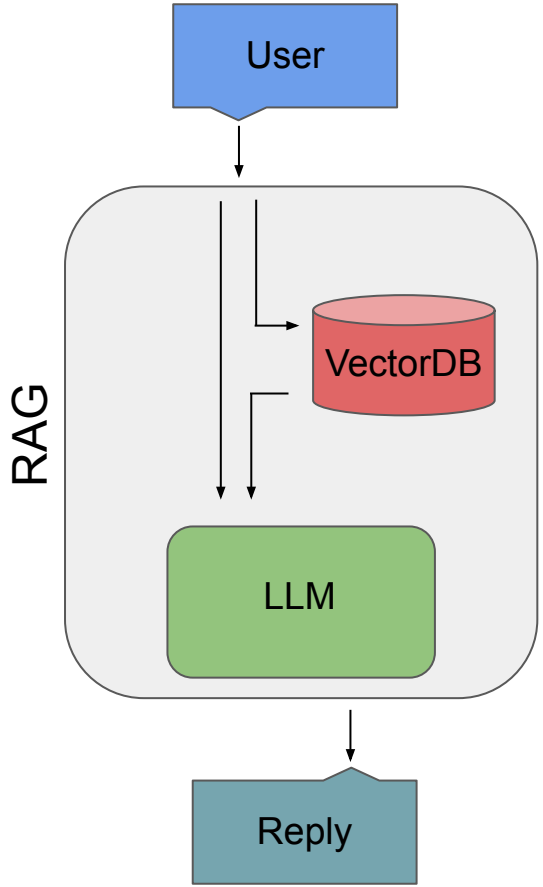


Retrieval Augmented Generation (RAG)

- It is a way of supplementing a model to have recent information automatically provided alongside the prompt.
- This involves a multistep process.
    - **LangChain**, **LlamaIndex**, and many other tools can facilitate this.
- The database (vector store) of information is ***very*** important.



https://radical.vc/radical-reads-large-language-models/


User
RAG
VectorDB
LLM
Reply

LlamaIndex

LOGIN BOOK A DEMO

MAY 29, 2025 ▪ [LLAMACLOUD] [RAG] [AGENTS] [AGENTIC DOCUMENT WORKFLOWS]

RAG is dead, long live agentic retrieval

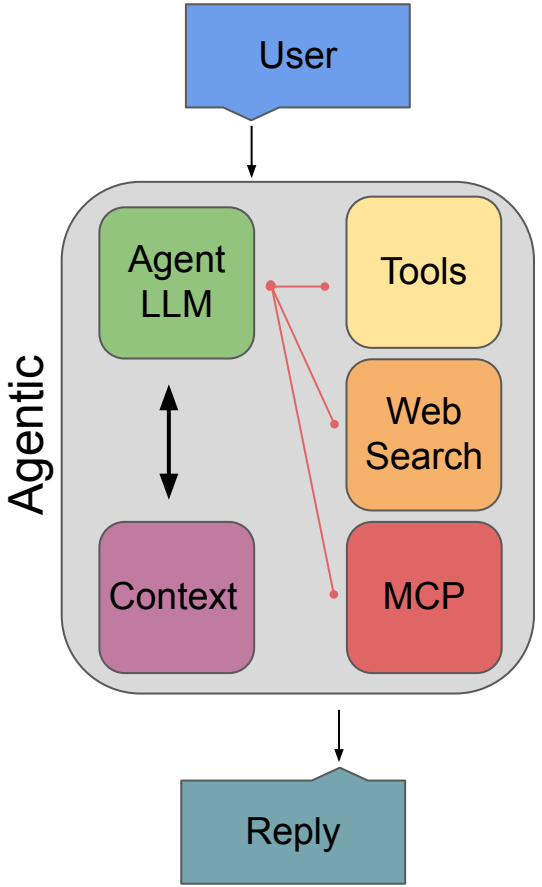
BY  SOURABH DESAI

So why did RAG “die”?

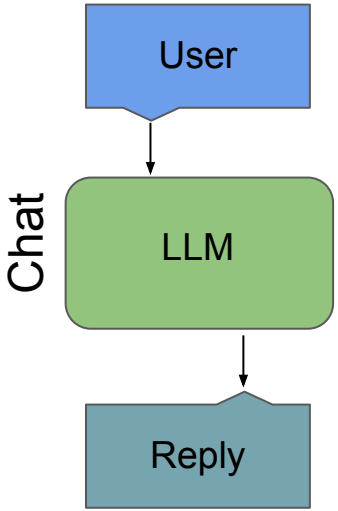
- Lots of shifts were happening in parallel.
- The **context** windows of models got *much* larger.
  - From ~16k tokens to >128k tokens. 256k - 1 million for frontier models.
  - To “talk” with an article, the entire article can just be passed as context now.
- “**Tool**” interfaces became much more common and useful.
  - **Tools** are functions or programs that the LLM can choose to run.
- A standard method to incorporate different sources into LLMs was developed.
  - The **M**odel **C**ontext **P**rotocol (**MCP**), slash-command `/skills`.

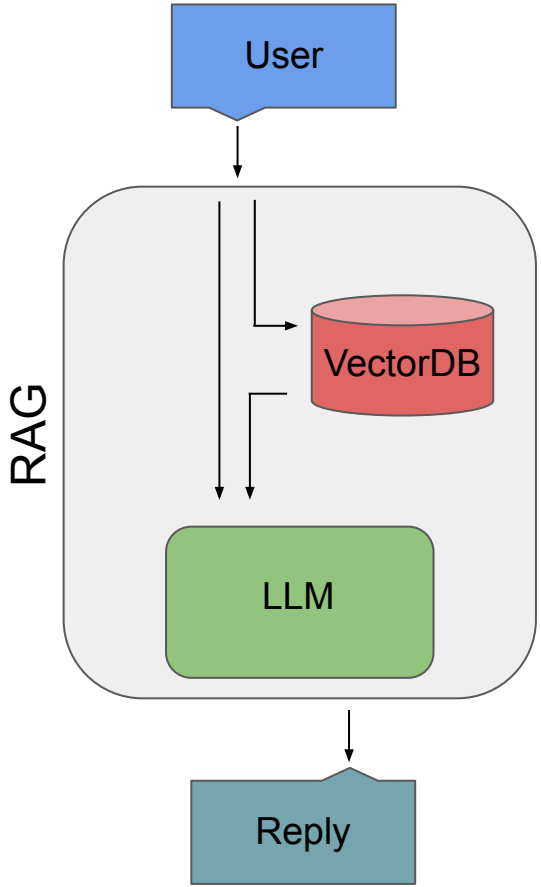
What is Agentic Retrieval?

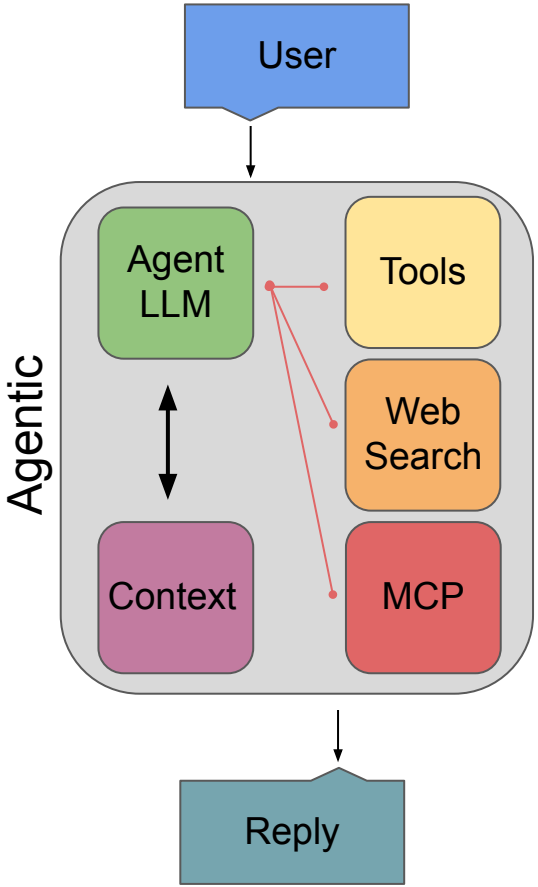
- It is a programmatic way of assembling context from different sources.
- **R**etrieval **A**ugmented **G**eneration (upper case) implies:
  - Only vector stores.
  - One loop per query.
- Agentic Retrieval is a more modular and composable *retrieval augmented generation* (lower case).



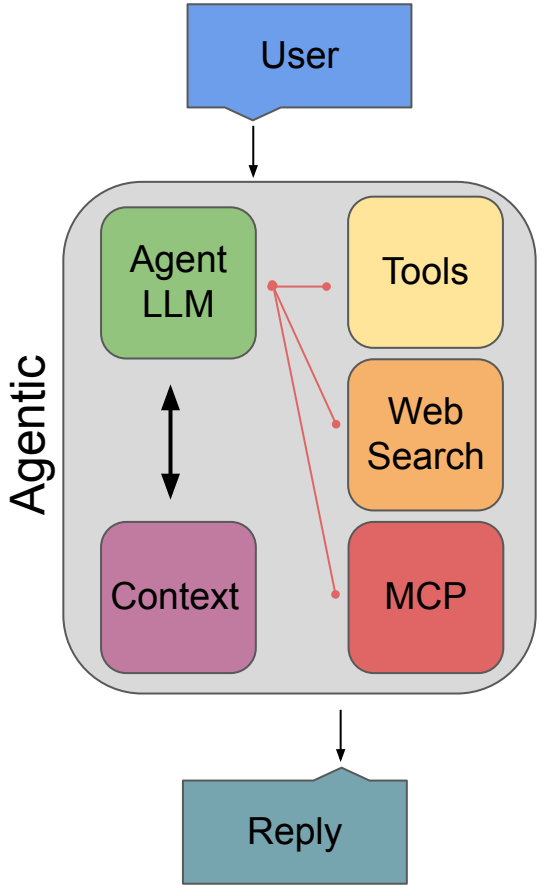
LLM vs. RAG vs. Agentic

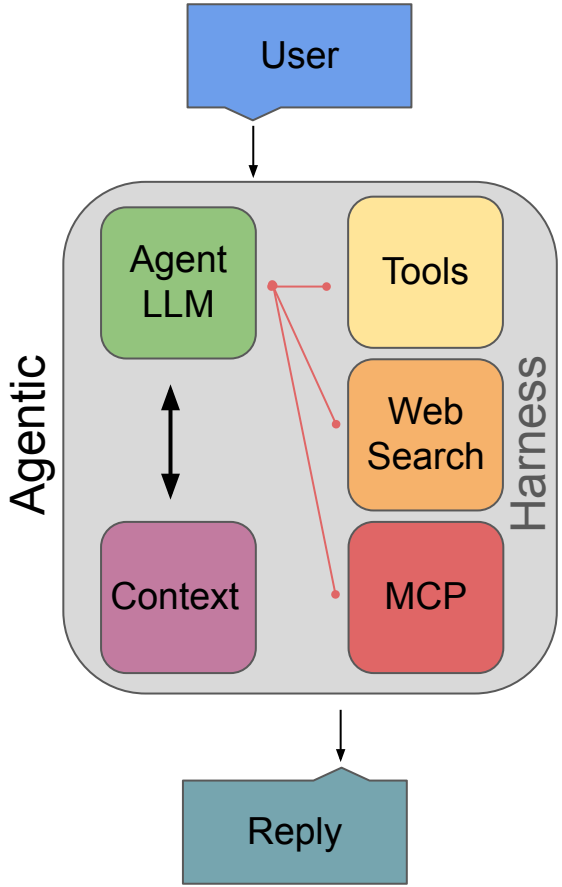




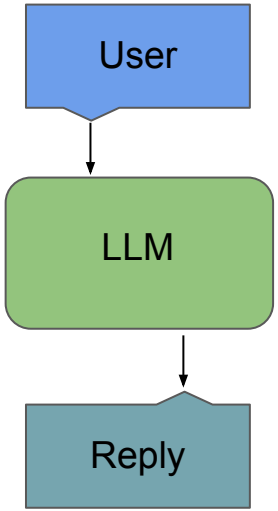


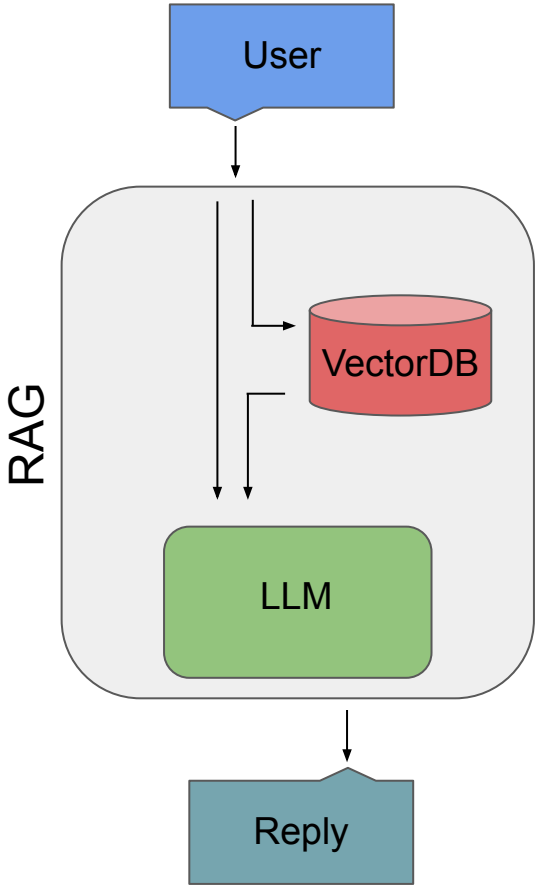
“Harnessing” AI Agents for Data Science

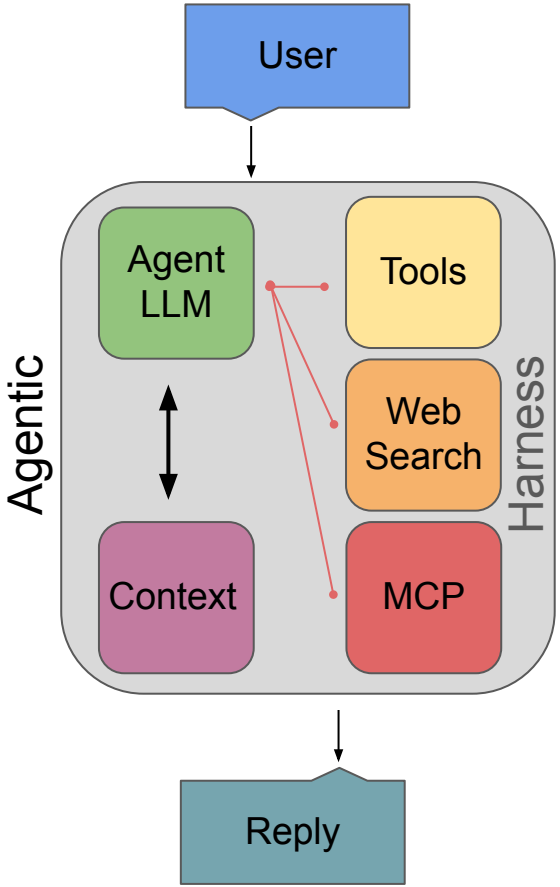

User
Agent
LLM
Tools
Web
Search
Agentic
Context
MCP
Reply

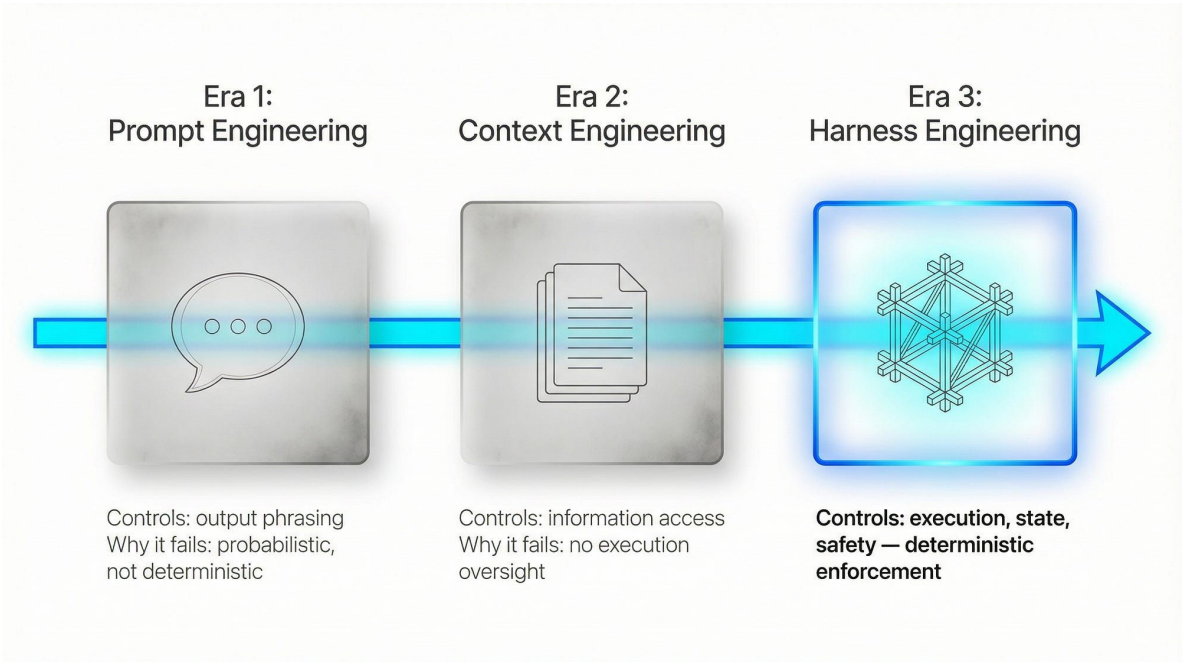

User
Agent
LLM
Tools
Web
Search
Agentic
Harness
Context
MCP
Reply

LLM vs. RAG vs. Agentic Harness

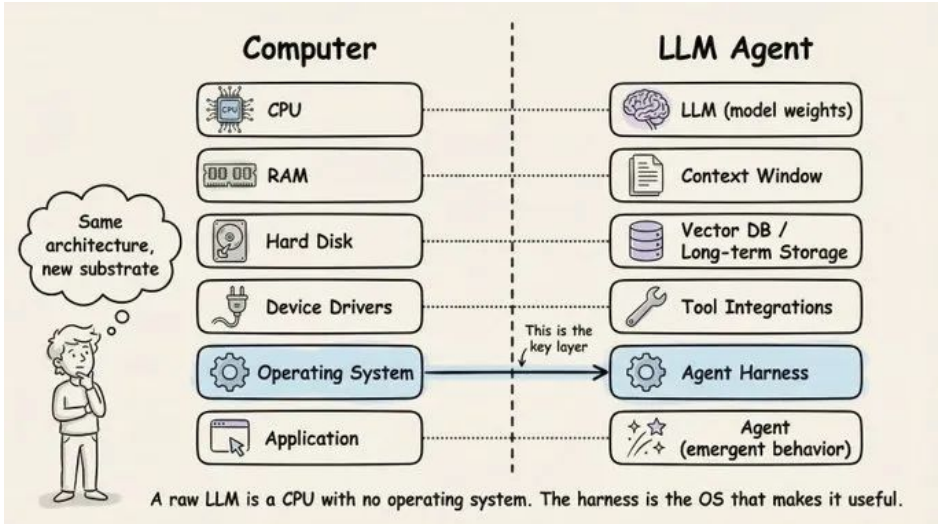








https://medium.com/@wasowski.jarek/ai-agent-harness-architecture-7-patterns-that-control-autonomous-agents-in-production-d07a94a9cdcd



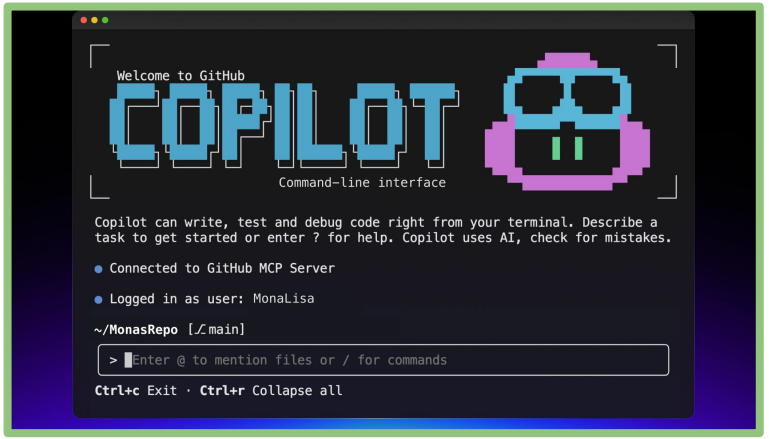
https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness

CLAUDE CODE

opencode

The open source AI coding agent

CLAUDE CODE

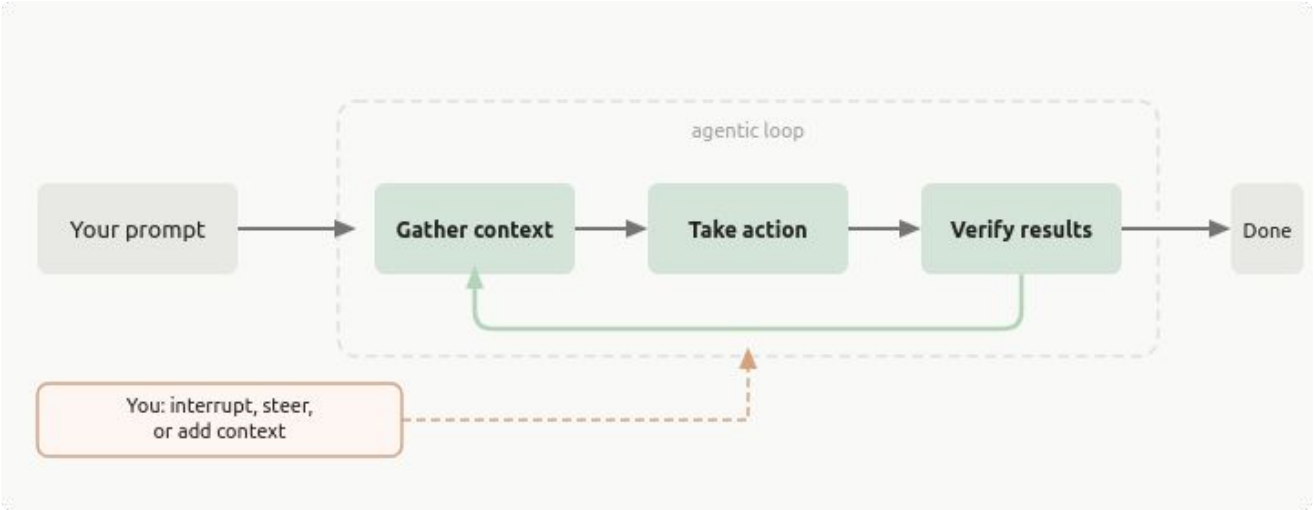


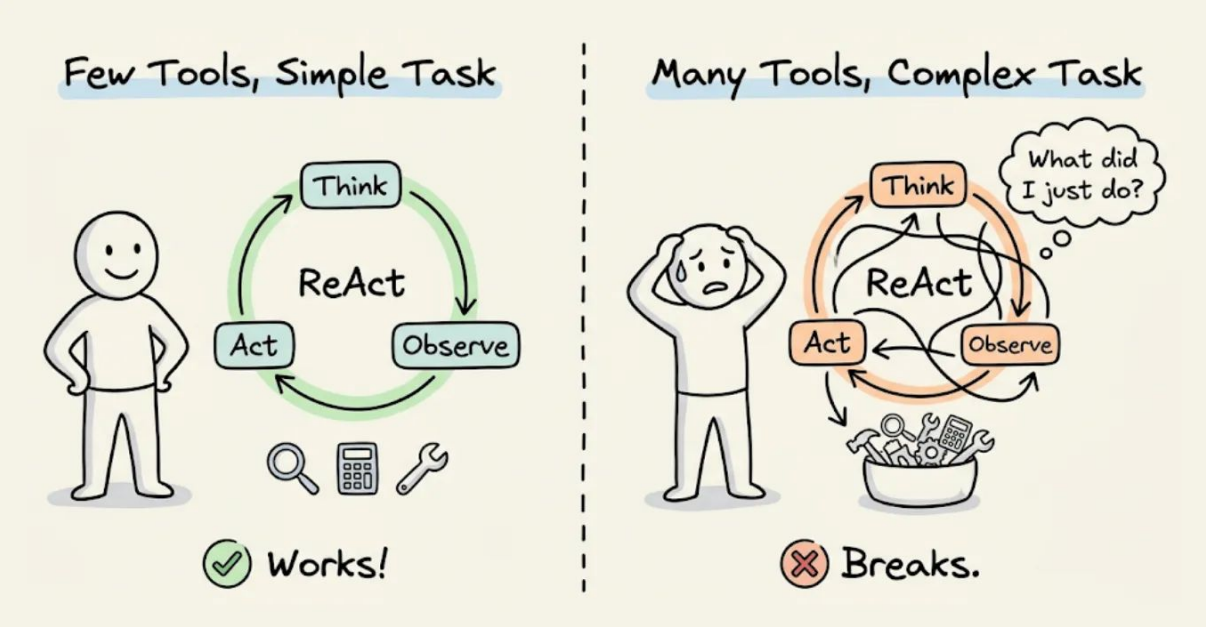
opencode

The open source AI coding agent

Agentic “Harness”

An agentic harness is: the software infrastructure—separate from the AI model itself—that governs how an agent operates, managing its lifecycle, memory, and interactions with external tools.

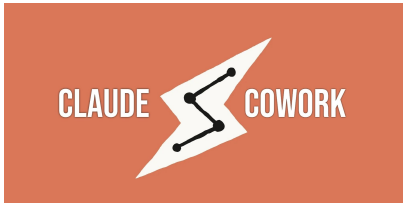




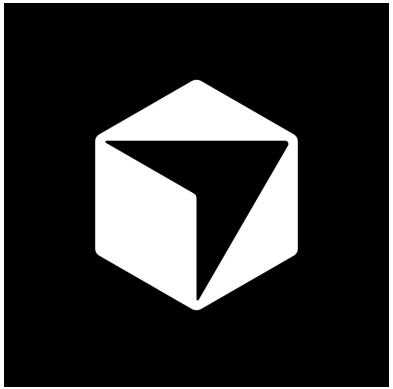
https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness

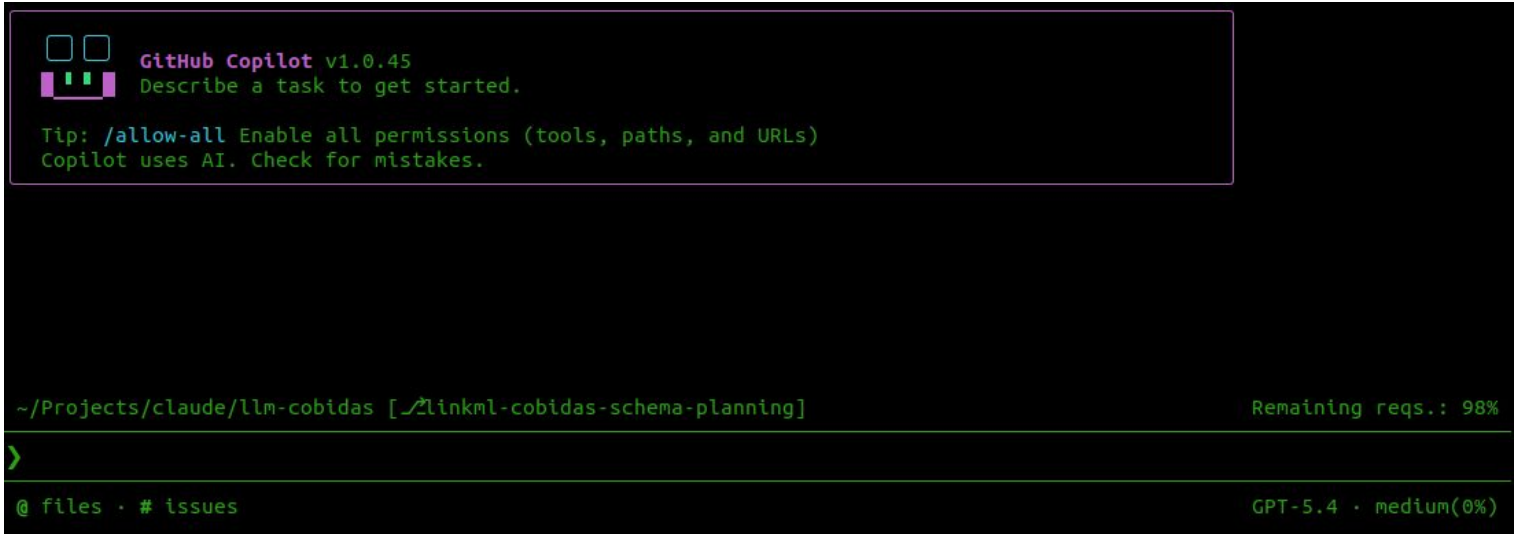
A Web Interface vs. a “Local” Interface

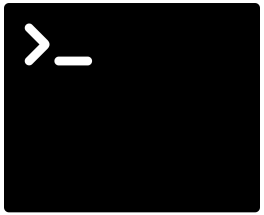
- There is an increasing shift toward moving your workflows to an online platform or a local application.
  - This does allow for your long-running tasks to happen in the background
- However, for depending on the kinds of work this may not be viable.
  - For large datasets, this is probably not practical. This will never be viable if privacy is required.
  - Coding projects often require local management and environments that are challenging online.
- Even “local” interfaces send data from your machine, so discretion is needed.

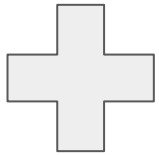


Integrate Directly into your IDE / Editor




GitHub Copilot v1.0.45
Describe a task to get started.
Tip: /allow-all Enable all permissions (tools, paths, and URLs)
Copilot uses AI. Check for mistakes.
~/Projects/claude/llm-cobidas [⎇linkml-cobidas-schema-planning]
Remaining reqs.: 98%
❯
@ files · # issues
GPT-5.4 · medium(0%)



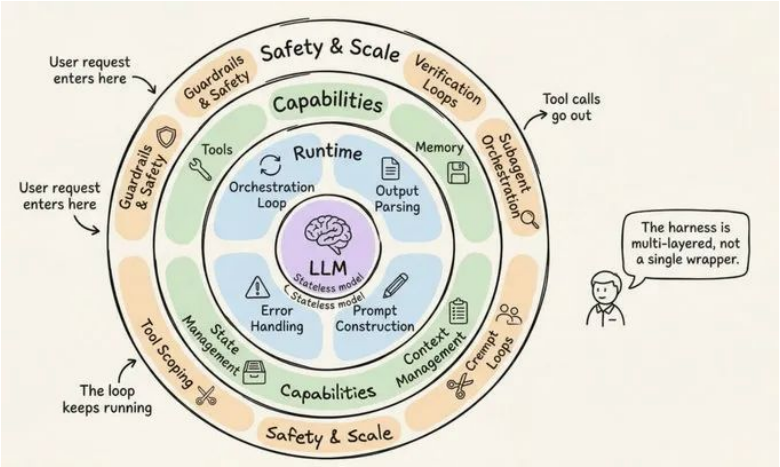



tmux

Core Configuration

Agentic Harness - Core Configuration

1. Context / State Management
   a. What tools, prompts, instructions, etc. are most relevant
   b. What has been loaded into the LLM (system calls), available context, etc.
2. Control Loop
   a. Modes
   b. Settings / CLAUDE.md / AGENTS.md
   c. Hooks
3. Tools
   a. Commands and `/skills`
   b. Subagents - Multi-agent Workflows
   c. Tools / MCP / Plugins
4. Memory
   a. Keep track of your plans and work.

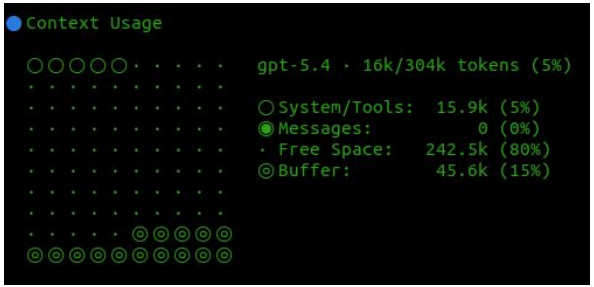


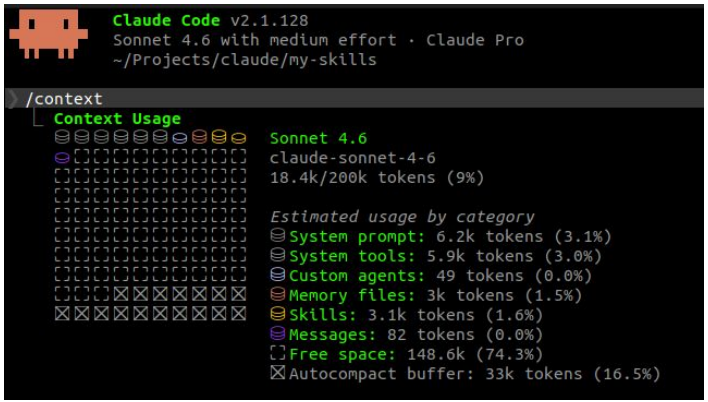
https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness

Agentic Harness - Context / State

Agentic Harness - Context Management

- The **context** available in the harness is the source / reference files, tools, skills, etc. that can be loaded into memory.
  - ~250k tokens - but just the prompt and default context loading can take ~15k tokens.
- **IMPORTANT: Performance begins degrading after <u>50%</u> of context is filled.**





Agentic Harness - State Management

- The **state** of the agent harness is the full provenance and order of what has been loaded into the model's context.
- Some tools let you resume from specific "states", but this isn't always available.
- Depending on some model settings, it may be impossible to perfectly recover a **state**.
- It is a good idea to treat **states** as ephemeral. Once the agent has a clear understanding, have it save everything it might need to recover the idea.

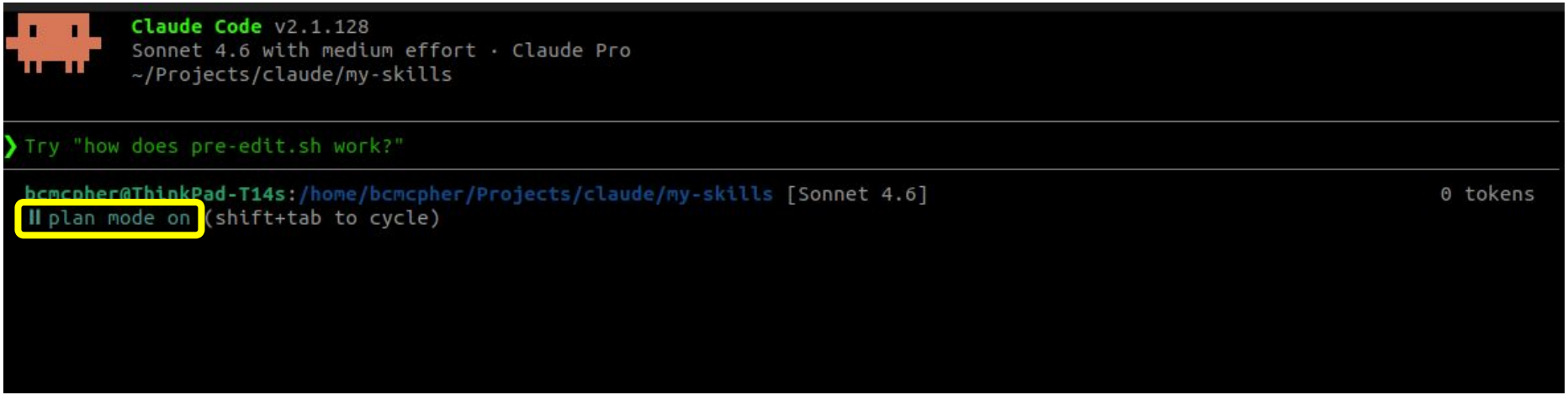
Agentic Harness - Context / State Management

- The available **context** for a given agent indicates how much more information it can incorporate and “thinking” it can do before it has to `/compact`.
- **Compaction** involves the model writing a summary of everything in its **state**, clearing everything in its memory, then re-loading it’s default **context** before loading the summary of the previous **state**.
    - **This can lead to the loss of specific instructions established in your plan!**
- How you have configured your **Settings**, **Plan**, `/skills`, `@subagents`, and **memory** can greatly impact how much you can accomplish within a session.

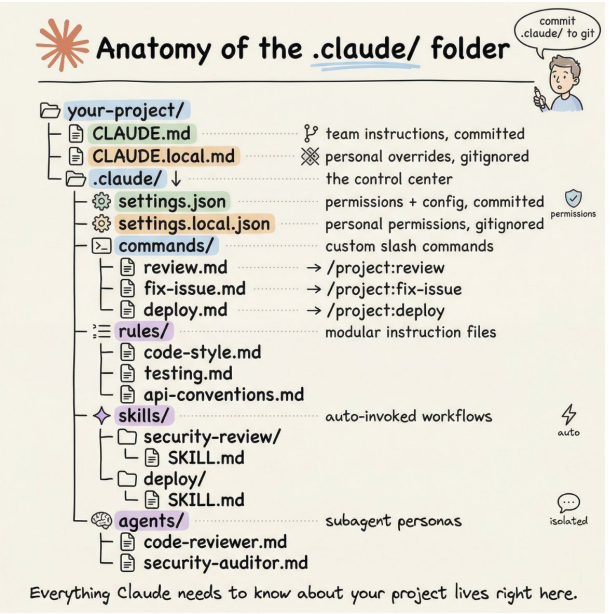
Agentic Harness - Control Loop

Agentic Harness - Control Loop and Modes

- Agents have an internal loop for managing their interactions with context, tools and “thoughts”.
- This is an iterative cycle of augmenting and clarifying instructions before the agent(s) take actions.
  - Provides the ability to orchestrate different multi-agent workflows.
- Most harnesses have global modes in which prompts are interpreted.
  - Typically **Plan** and **Edit** modes (some have a **Query** / **Chat** mode).
  - Generally you should always start in **Plan** mode.


Claude Code v2.1.128
Sonnet 4.6 with medium effort · Claude Pro
~/Projects/claude/my-skills
❯ Try "how does pre-edit.sh work?"
bcmcpher@ThinkPad-T14s:/home/bcmcpher/Projects/claude/my-skills [Sonnet 4.6]
0 tokens
⏸ plan mode on (shift+tab to cycle)

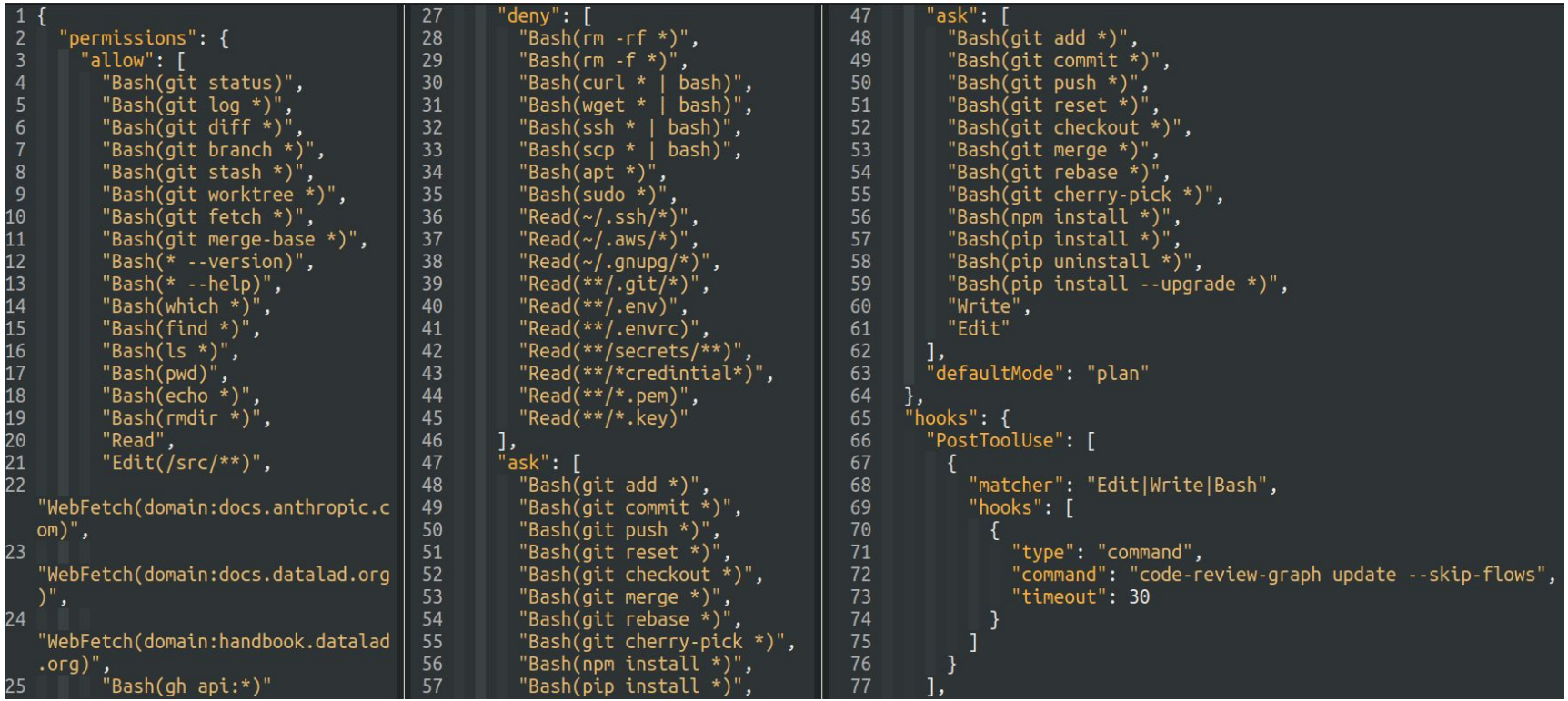
Agentic Harness - Settings



- Settings are hierarchically loaded.
  - Your global settings in `$HOME` (`~/`)
  - The root folder of your codebase
  - Any folder within your codebase.
- Settings load in global -> local priority and **append**.
- You can override settings with local config changes, but settings defined further up the hierarchy remain.
- Generally, these are version controlled (git tracked).
  - Add `*.local.*` infix to keep system specific overrides. (e.g. settings.local.md, etc.)

Agentic Harness - Settings

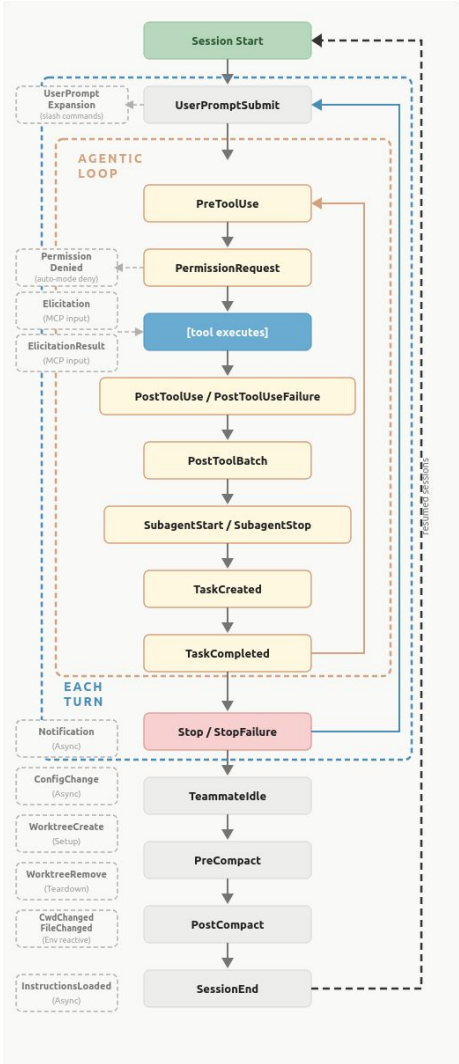
- Most tools have some kind of settings file for permissions.
  - Configure `allow` / `ask` / `deny` patterns around common operations immediately.
- This may take some iterations.
  - If you too aggressively prevent some config files from being accessed, the agent won't know you have a working, configured environment.
- There are many global settings, and they are often adding more.
  - It is worth keeping up to date and exploring what you can customize.



```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git log *)",
      "Bash(git diff *)",
      "Bash(git branch *)",
      "Bash(git stash *)",
      "Bash(git worktree *)",
      "Bash(git fetch *)",
      "Bash(git merge-base *)",
      "Bash(* --version)",
      "Bash(* --help)",
      "Bash(which *)",
      "Bash(find *)",
      "Bash(ls *)",
      "Bash(pwd)",
      "Bash(echo *)",
      "Bash(rmdir *)",
      "Read",
      "Edit(/src/**)",
      "WebFetch(domain:docs.anthropic.com)",
      "WebFetch(domain:docs.datalad.org)",
      "WebFetch(domain:handbook.datalad.org)",
      "Bash(gh api:*)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(rm -f *)",
      "Bash(curl * | bash)",
      "Bash(wget * | bash)",
      "Bash(ssh * | bash)",
      "Bash(scp * | bash)",
      "Bash(apt *)",
      "Bash(sudo *)",
      "Read(~/.ssh/*)",
      "Read(~/.aws/*)",
      "Read(~/.gnupg/*)",
      "Read(**/.git/*)",
      "Read(**/.env)",
      "Read(**/.envrc)",
      "Read(**/secrets/**)",
      "Read(**/*credintial*)",
      "Read(**/*.pem)",
      "Read(**/*.key)"
    ],
    "ask": [
      "Bash(git add *)",
      "Bash(git commit *)",
      "Bash(git push *)",
      "Bash(git reset *)",
      "Bash(git checkout *)",
      "Bash(git merge *)",
      "Bash(git rebase *)",
      "Bash(git cherry-pick *)",
      "Bash(npm install *)",
      "Bash(pip install *)",
      "Bash(pip uninstall *)",
      "Bash(pip install --upgrade *)",
      "Write",
      "Edit"
    ],
    "defaultMode": "plan"
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write|Bash",
        "hooks": [
          {
            "type": "command",
            "command": "code-review-graph update --skip-flows",
            "timeout": 30
          }
        ]
      }
    ],
```

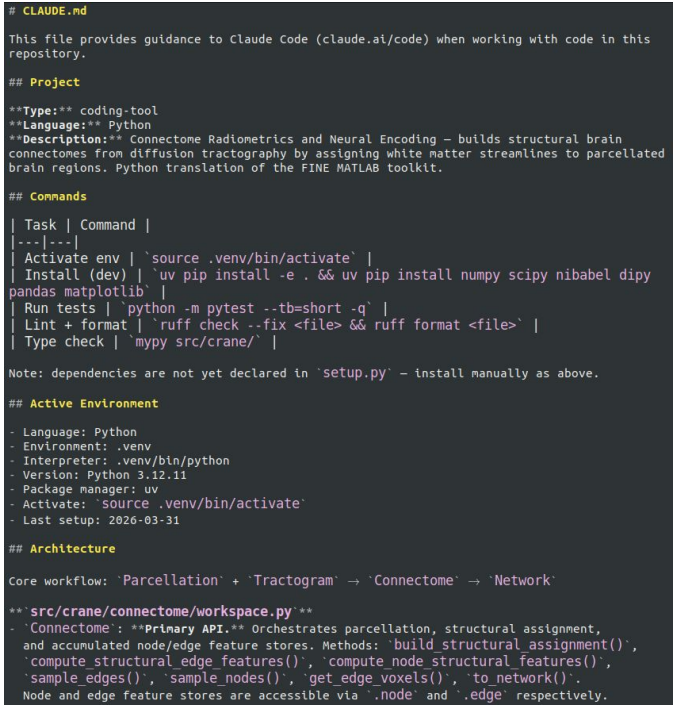
Agentic Harness - Hooks

- Hooks provide a way to programmatically run checks on the Agents I/O as it's working.
- Examples:
  - **PreToolUse**: Verify that protected files aren't being changed.
  - **PostToolUse**: Lint / Type check syntax files.
  - **TaskCompleted**: `datalad save`



Agentic Harness - CLAUDE.md / AGENTS.md

- CLAUDE.md / AGENTS.md provide a summary of the project for the agents.
  - It is always loaded into context.
  - This is often not efficient and can easily cause more harm than good.
- There are mixed findings on the benefits of including this file at all.
  - Keep it minimal and see what works for you.



```
# CLAUDE.md

This file provides guidance to Claude Code (claude.ai/code) when working with code in this
repository.

## Project

**Type:** coding-tool
**Language:** Python
**Description:** Connectome Radiometrics and Neural Encoding — builds structural brain
connectomes from diffusion tractography by assigning white matter streamlines to parcellated
brain regions. Python translation of the FINE MATLAB toolkit.

## Commands

| Task | Command |
|---|---|
| Activate env | `source .venv/bin/activate` |
| Install (dev) | `uv pip install -e . && uv pip install numpy scipy nibabel dipy
pandas matplotlib` |
| Run tests | `python -m pytest --tb=short -q` |
| Lint + format | `ruff check --fix <file> && ruff format <file>` |
| Type check | `mypy src/crane/` |

Note: dependencies are not yet declared in `setup.py` — install manually as above.

## Active Environment

- Language: Python
- Environment: .venv
- Interpreter: .venv/bin/python
- Version: Python 3.12.11
- Package manager: uv
- Activate: `source .venv/bin/activate`
- Last setup: 2026-03-31

## Architecture

Core workflow: `Parcellation` + `Tractogram` → `Connectome` → `Network`

**`src/crane/connectome/workspace.py`**
- `Connectome`: **Primary API.** Orchestrates parcellation, structural assignment,
  and accumulated node/edge feature stores. Methods: `build_structural_assignment()`,
  `compute_structural_edge_features()`, `compute_node_structural_features()`,
  `sample_edges()`, `sample_nodes()`, `get_edge_voxels()`, `to_network()`.
  Node and edge feature stores are accessible via `.node` and `.edge` respectively.
```

Agentic Harness - Tools

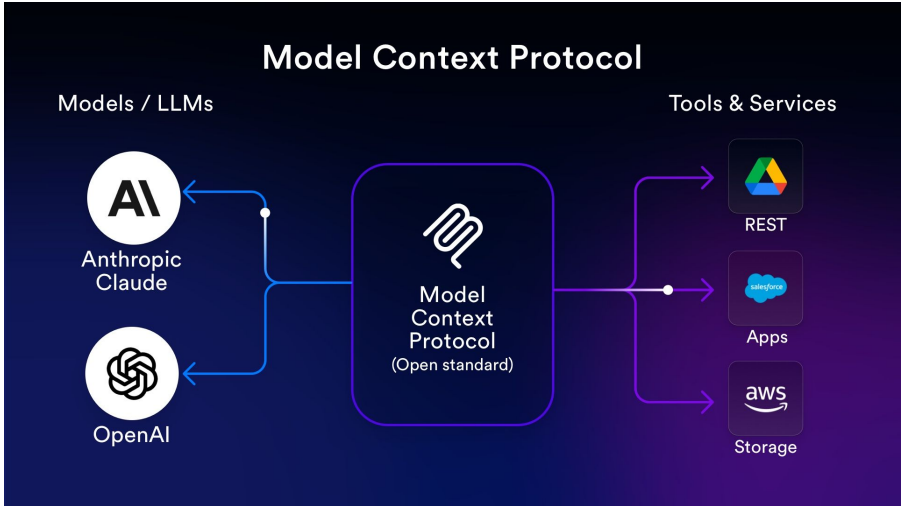
Agentic Harness - Tools

- The actions a harness makes available to gain context or modify files.
  - You get some oversight with these from modifying settings.
- They can be for collecting context:
  - **Read** - plain text, pdf?, docx?
  - **WebFetch / WebSearch**
  - `Bash`: `ls`, `pwd`, `find`, `grep`, `wc`, etc.
- They can be for making changes
  - **Edit** / **MultiEdit** plain test files.
  - `Bash`: `rm`, `wget`, `curl`, `git`, `gh`, `uv` / `pip`, `npm`, etc.
- Generally a few dozen **Tools** are available by default (+ many `bash` commands).
  - You can add more with customizations or by connecting a **Model Context Protocol** (**MCP**).

```
Agent
AskUserQuestion
Bash
CronCreate
CronDelete
CronList
Edit
EnterPlanMode
EnterWorktree
ExitPlanMode
ExitWorktree
Glob
Grep
ListMcpResourcesTool
LSP
Monitor
NotebookEdit
PowerShell
Read
ReadMcpResourceTool
SendMessage
ShareOnboardingGuide
Skill
TaskCreate
TaskGet
TaskList
TaskOutput
TaskStop
TaskUpdate
TeamCreate
TeamDelete
TodoWrite
ToolSearch
WebFetch
WebSearch
Write
```

Model Context Protocol (MCP)

- Introduced by Anthropic, the MCP is a standardized way for LLM applications to communicate with and access external tools, data, and services.
- Compartmentalizes the tooling needed for agents to add tools for external functions and services.



https://astrix.security/glossary/model-context-protocol-mcp/

Agentic Harness - MCP

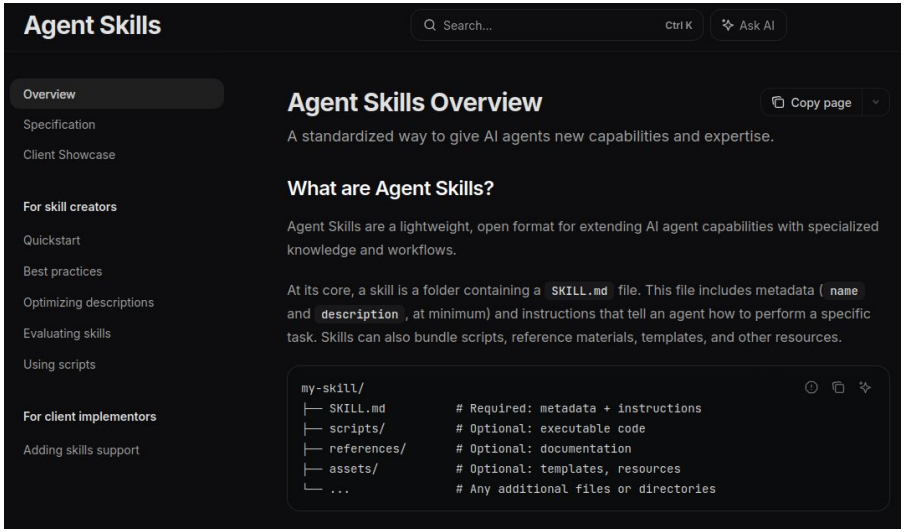
- Adds external **Tools** to an Agent / Harness.
- Unfortunately, new tools grow the default used context, so adding too many can erode reliability. You can overwhelm your agent with choice.
    - It is important to choose the tools you add judiciously.
- MCP are still the safest way to connect external tools with your session when authentication is required (Github, etc.)
- It is popular now to write `/skills` that document the integration you want.
    - This is easier to maintain and generally takes less context to deploy (for now).

Agentic Harness - Commands (`/skills 1.0`)

- These are markdown files meant to provide reusable prompts and instructions.
  - There is some functionality for argument passing or custom instructions.
  - However, they are generally meant to just be invoked.
- They are invoked with a leading `/` , leading to their original moniker of **slash-commands**.
- While these are still supported, they've mostly been superseded by `/skills`.

Agentic Harness - `/skills (2.0)`

- Skills are preconfigured sets of instructions that a user can invoke to complete different tasks.
- They are often bundled together based on expected uses.
- They can dynamically load `references/`, `assets/`, or `scripts/` files.
- The layout of skills is emerging as an independent standard.



https://agentskills.io/home

Agentic Harness - `/skills`

- Compared to commands, `/skills` have more nuance in their invocation.
  - They can be assigned trigger phrases and other context about when they should be used.
  - They can be directly invoked or contextually invoked.
- The shared resource files can be dynamically loaded by the skills to provide further contextual information.
  - It helps manage context usage and minimizes redundant information.
- Anthropic has `/skill-creator` which is useful for refining the structure of your skills into this standard.
  - Help you maintain good practices in their composition and develop tests for verification.

```
---
name: datalad-configuration
description: >
  Auto-invoke when the user wants to read or change DataLad or git-annex configuration
  within a dataset — e.g., setting annex backend, configuring credential helpers,
  adjusting dataset-level variables, or inspecting current config values. Trigger on
  "configure the dataset", "set annex backend", "check dataset config", "set datalad
  config", "change git config in this dataset", "configure credential", or
  /datalad-configuration. Do NOT trigger for global git config unrelated to DataLad.
argument-hint: [get|set|unset] [section.key] [value] [--scope dataset|local|global]
user-invocable: true
disable-model-invocation: false
allowed-tools: Read, Bash, Glob
---

# Skill: datalad-configuration

Read and write dataset-scoped, clone-local, or global DataLad/git configuration using
`datalad configuration`. This is the DataLad-aware wrapper around `git config` that
handles DataLad-specific keys and scopes correctly.

## Scopes

| Scope | Where stored | Effect |
|-------|--------------|--------|
| `dataset` | `.datalad/config` (committed) | Applies to all clones of this dataset |
| `local` | `.git/config` (not committed) | Applies only to this clone |
| `global` | `~/.gitconfig` | Applies to all repos for this user |

Use `dataset` scope for settings that should travel with the dataset (e.g., annex
backend choice). Use `local` for clone-specific overrides (e.g., credential paths).

## Steps

1. **Verify DataLad context** — check for `.datalad/` in the current directory:
   ```bash
   ls .datalad/ 2>/dev/null
   ```
   Configuration can still be inspected/set globally even outside a dataset, but warn
   the user if no dataset is found.

2. **Determine action** — from `$ARGUMENTS` or conversation context:
   - **get**: read the current value of a config key
   - **set**: write a new value
   - **unset**: remove a key
```

Agentic Harness - `@subagents`

- `@subagents` are scoped, independent processes that can be deployed by the harness to accomplish specific tasks.
- `@subagents` maintain their own **context**.
  - This is useful for managing your overall context for complicated tasks.
  - Subagents can compartmentalize attention on narrowly defined tasks.
- `@subagents` can have prompts refined to specialized workflows alongside access to dedicated tools, `/skills`, etc. not available in the general context.
- The moniker of “harness” comes from the idea of harnessing a team of subagents to coordinate complex tasks.

Agentic Harness - `@subagents`

- Subagents defined as markdown files (.md) similar to `/skills`.
- They can be launched as your primary scope.
- The real advantage is being able to coordinate a group of subagents to manage different parts of a large task.

```
---
name: code-reviewer
description: Expert code review specialist. Proactively reviews code for quality, security, and
maintainability. Use immediately after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---

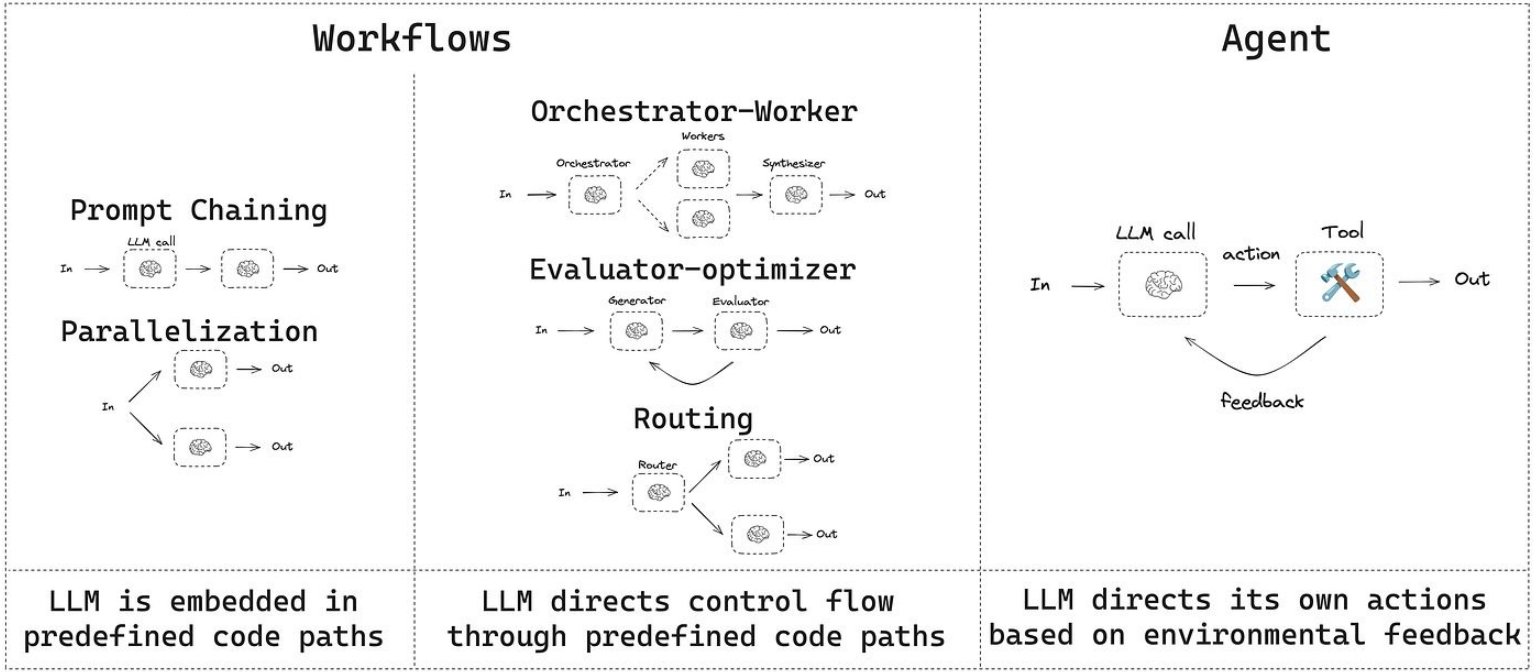
You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:
1. Run git diff to see recent changes
2. Focus on modified files
3. Begin review immediately

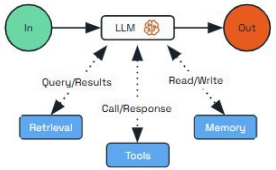
Review checklist:
- Code is clear and readable
```

Agentic Harness - `@subagent` Workflows

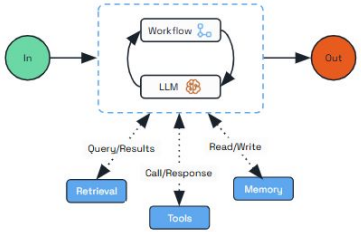
- There are many workflows for curating agents to accomplish tasks.



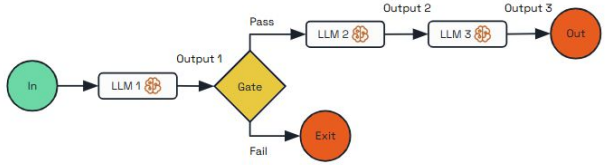
Augmented LLM Pattern



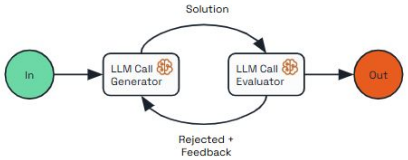
Durable Agent Pattern



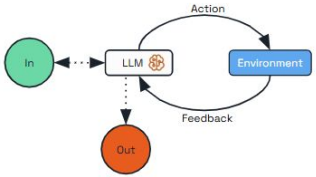
Prompt Chaining Pattern



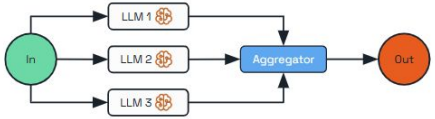
Evaluator Optimizer Pattern



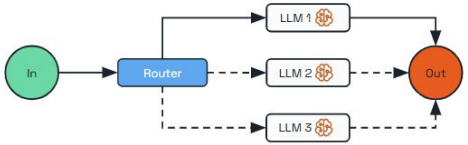
Autonomous Agent Pattern



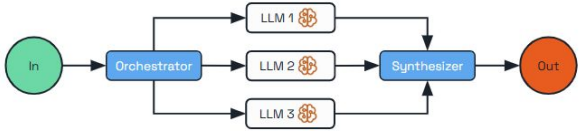
Parallelization Pattern



Routing Pattern

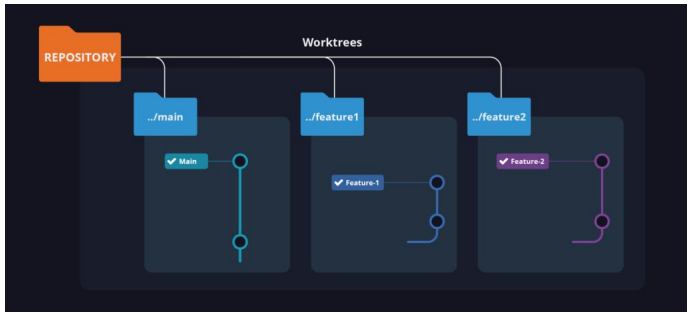


Orchestrator Workers Pattern



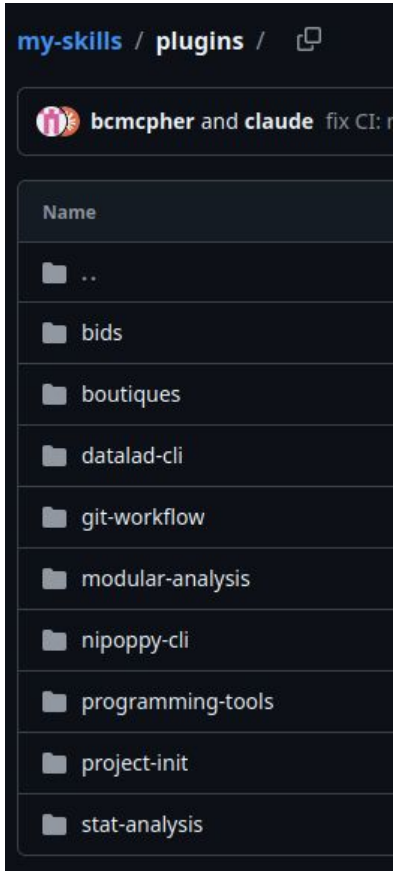
Git Worktree

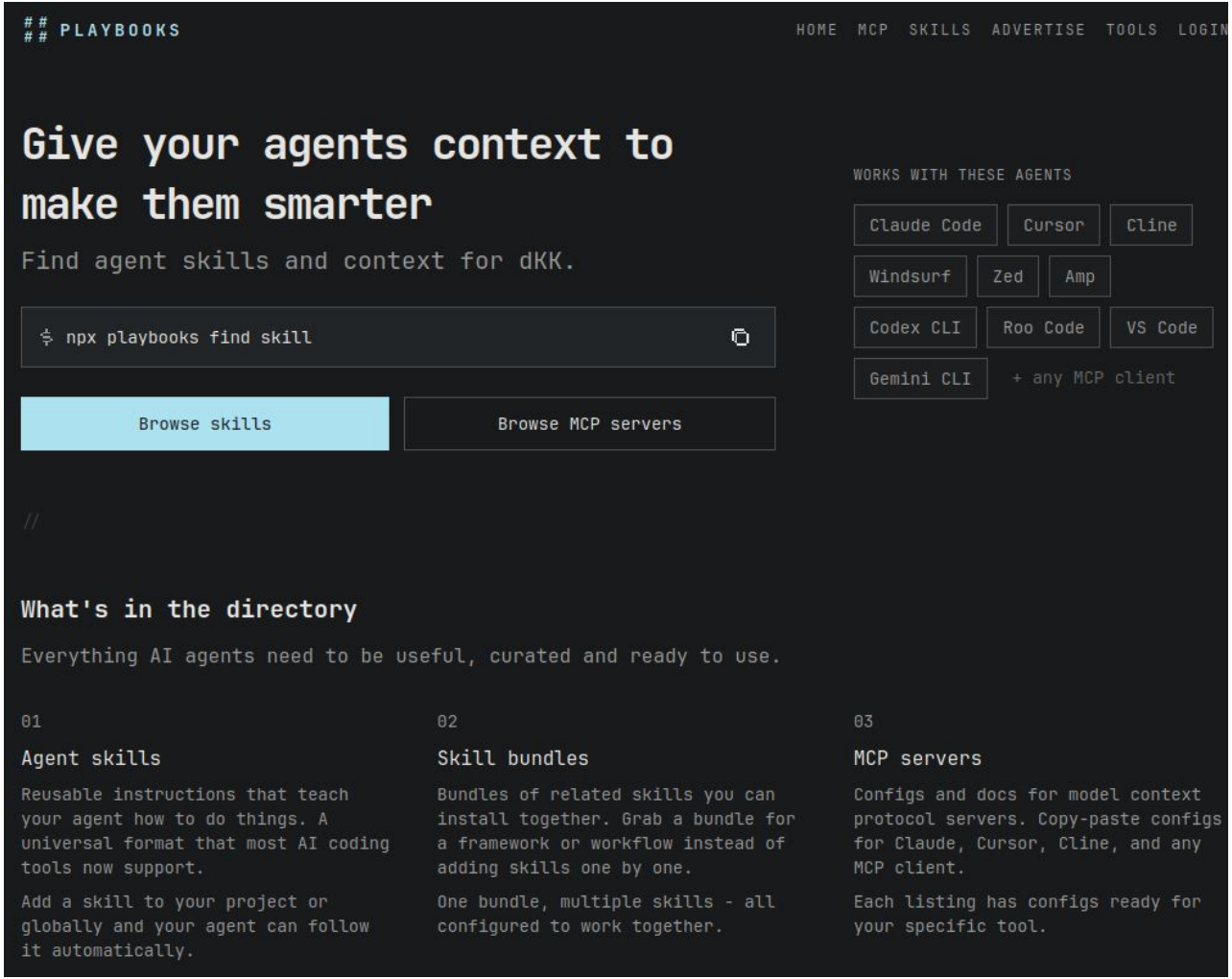
- `@subagents` allow for parallel work.
  - However, a git repo can only have 1 branch edited at a time.
- `git worktree` is a built-in feature to handle simultaneous edits on multiple branches. The harness can invoke and manage this for you.
- It effectively creates a local copy of the repo per branch that can be easily merged (assuming no conflicts) once work is completed.



Agentic Harness - Plugins

- A plugin is just a configured collection of `/skills`, `@subagents`, MCPs, etc. that can be installed as a collection.
- Because all of these can be installed either globally or by project, you can easily curate the functionality you want.




PLAYBOOKS
HOME MCP SKILLS ADVERTISE TOOLS LOGIN
Give your agents context to make them smarter
Find agent skills and context for dKK.
$ npx playbooks find skill
Browse skills
Browse MCP servers
WORKS WITH THESE AGENTS
Claude Code
Cursor
Cline
Windsurf
Zed
Amp
Codex CLI
Roo Code
VS Code
Gemini CLI
+ any MCP client
What's in the directory
Everything AI agents need to be useful, curated and ready to use.
01
Agent skills
Reusable instructions that teach your agent how to do things. A universal format that most AI coding tools now support.
Add a skill to your project or globally and your agent can follow it automatically.
02
Skill bundles
Bundles of related skills you can install together. Grab a bundle for a framework or workflow instead of adding skills one by one.
One bundle, multiple skills - all configured to work together.
03
MCP servers
Configs and docs for model context protocol servers. Copy-paste configs for Claude, Cursor, Cline, and any MCP client.
Each listing has configs ready for your specific tool.

Agentic Harness - Memory

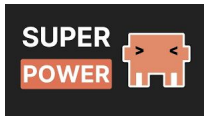
Agentic Harness - Memory

- Each harness has some mechanism for tracking memories based on your decisions and workflows.
  - They will gradually learn your habits based on observed usage.
  - However, they are not really meant for your long term plans or designs.
- It is worthwhile to regularly save plans for different implementations to disk.
  - This will save time (and tokens) when you can load and resume an in progress plan instead of regenerating the entire design from scratch.
- There are whole frameworks you can adopt to manage this process.

Agentic Harness - Memory

- **Spec Driven Development** - define an overarching goal for your project, then propose and develop tasks to complete them. A “waterfall” approach.
  - spec-kit, openspec, etc.

- **Agile development** - define a narrowly scoped, articulated improvement to the code as it currently exists.
  - superpowers, GSD, etc.



- **Atomic tasks** - just keep restating and iterating on a topic until it’s done. Take two steps forward, one step back, repeat.
  - Plan Mode, Ralph Loop.

Agentic Harness - Memory

- **Spec Driven Development** - Takes more up front planning (waterfall) and tracking, but you can seamlessly resume tasks. However, it can be very challenging to incorporate into existing projects.
- **Agile development** - More flexible for making modular changes and capable of dealing with the code as it is - but you have to regenerate context for every change.
- **Atomic tasks** - Simple to deploy, but it might take (significantly) more iterations to create a working product than you expect.

Example Configuration & Workflows

How’ve I’ve Harnessed Harnesses

There are 3 kinds of projects I’ve found myself repeatedly creating and refining:

1. **Code Repositories**: specifically for packaging previous work I’ve done into an installable python package that can has a general interface.
    1.1. I’m often converting from MATLAB to python
2. **Research Analyses**: restructuring analysis scripts, plotting functions, and reports into more easily audited and deployed scripts.
    2.1. It handles the refactoring of first draft analysis and plotting scripts in a few passes.
3. **Textual Research**: aggregates and manages sources for planning, research, and writing.
    3.1. Local LLM Wiki (memex)

Always Configured

- Modify the settings.json (or equivalent) to set globally:
  - `allow` / `ask` / `deny` Permissions
  - Start in “**Plan**” mode
  - File protection hooks
- Add any `/skills` or MCP server you always want available.
  - Git / Github
  - Coding / Debugging agents and skills.

Always Configured

- Set up a `venv` / `conda` environment to track your dependencies.
- Add hooks for linting, type checking, etc. for the language you're working with.
- Add any specialized MCP servers for accessing external tools.
- Connect your LSP further code quality feedback.
- Select a default model / effort.
- Determine how you want to develop and track your project.
    - Spec-kit, OpenSpec, Superpowers, internally, etc.

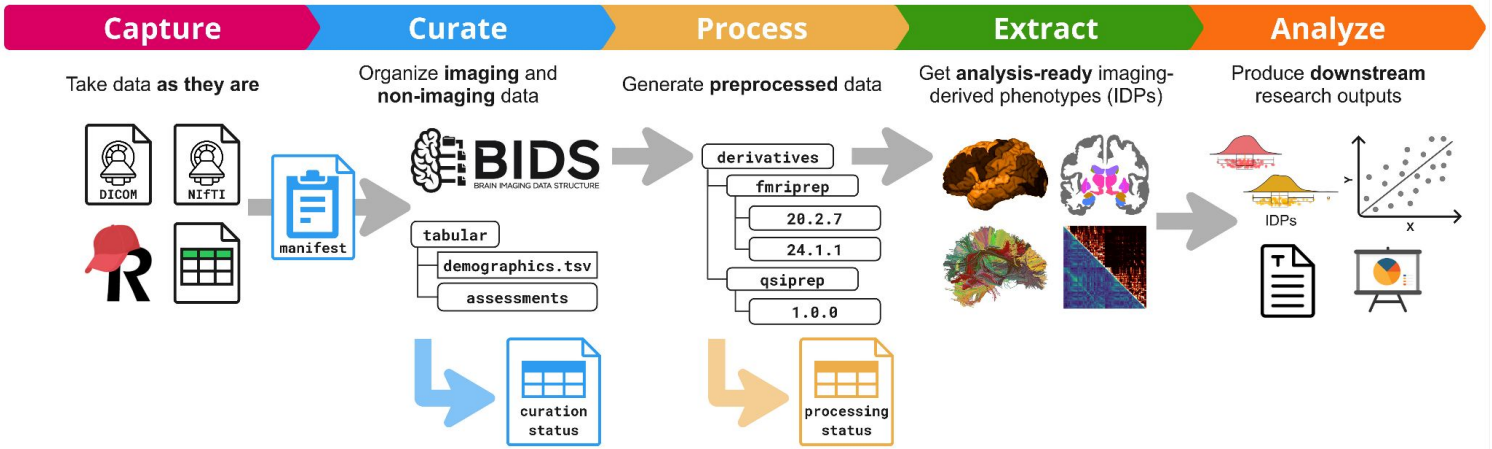
Code Repository Setup

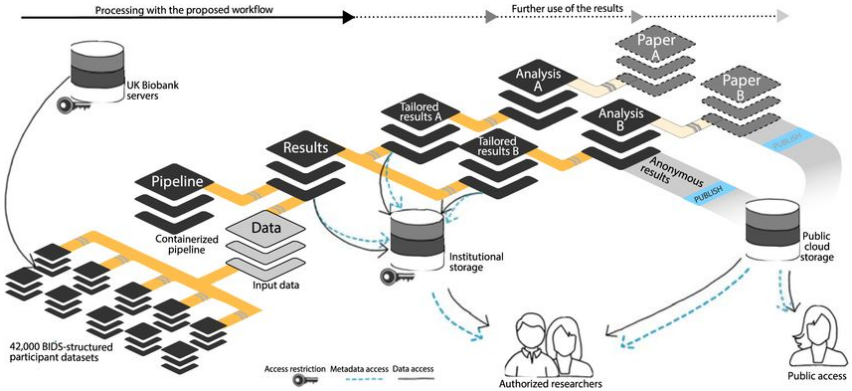
- Add other coding specific resources.
  - code-review-graph, `/skills` / `@subagents` / plugins, etc.
- Identify any good reference examples of structure or algorithms.
- Github Actions for CI / CD.

Research Analyses Setup

- Make sure your data are write-protected or backed up.
  - If something goes wrong you don’t want to lose your only working copy.
- Adjust the expectations of your coding agents for research.
  - Create single, linear files for analyses.
  - Favor interactive scripts / notebooks over modular, interdependent files.
  - Divide scripts into logically sequential chunks: data import, modelling, plotting, etc.
  - Lower the priority on creating unit tests (initially).
- Nipoppy or Datalad `/skills`?

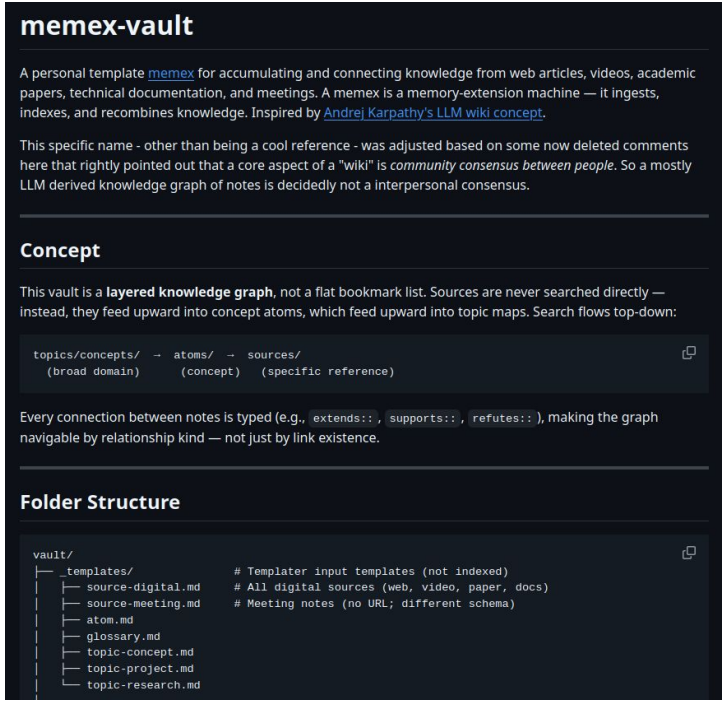
Nipoppy


Capture
Curate
Process
Extract
Analyze
Take data as they are
Organize imaging and non-imaging data
Generate preprocessed data
Get analysis-ready imaging-derived phenotypes (IDPs)
Produce downstream research outputs
DICOM
NIfTI
manifest
BIDS
BRAIN IMAGING DATA STRUCTURE
tabular
demographics.tsv
assessments
derivatives
fmriprep
20.2.7
24.1.1
qsiprep
1.0.0
IDPs
curation status
processing status


Processing with the proposed workflow
Further use of the results
UK Biobank servers
Pipeline
Containerized pipeline
Data
Input data
Results
Tailored results A
Tailored results B
Analysis A
Analysis B
Paper A
Paper B
Anonymous results
PUBLISH
Institutional storage
Public cloud storage
42,000 BIDS-structured participant datasets
Access restriction
Metadata access
Data access
Authorized researchers
Public access

Textual Research Setup

- Will this have any code involved, or just text?
  - You could track this alongside your research code…
- Determine a structure for tracking and exporting results.
  - Obsidian LLM Wiki
- Add MCP(s) for searching and export
  - PubMed, bioRxiv, and others have dedicated search functionality.
  - NotebookLM, GDrive, and others are available for structured exports.
- Your imagination is the limit of what this can handle.
  - Personal knowledge base of *anything*.
  - Literature Search for a research study.
  - Publish your documentation as a website.



memex-vault

A personal template memex for accumulating and connecting knowledge from web articles, videos, academic papers, technical documentation, and meetings. A memex is a memory-extension machine — it ingests, indexes, and recombines knowledge. Inspired by Andrej Karpathy's LLM wiki concept.

This specific name - other than being a cool reference - was adjusted based on some now deleted comments here that rightly pointed out that a core aspect of a "wiki" is *community consensus between people*. So a mostly LLM derived knowledge graph of notes is decidedly not a interpersonal consensus.

Concept

This vault is a **layered knowledge graph**, not a flat bookmark list. Sources are never searched directly — instead, they feed upward into concept atoms, which feed upward into topic maps. Search flows top-down:

```
topics/concepts/  →  atoms/  →  sources/
  (broad domain)      (concept)   (specific reference)
```

Every connection between notes is typed (e.g., `extends::`, `supports::`, `refutes::`), making the graph navigable by relationship kind — not just by link existence.

Folder Structure

```
vault/
├── _templates/              # Templater input templates (not indexed)
│   ├── source-digital.md    # All digital sources (web, video, paper, docs)
│   ├── source-meeting.md    # Meeting notes (no URL; different schema)
│   ├── atom.md
│   ├── glossary.md
│   ├── topic-concept.md
│   ├── topic-project.md
│   └── topic-research.md
```

Testimonials

I've had good luck with Claude Sonnet (4.6) Converting MATLAB code I've used previously into python. However, what I think happens is: Claude summarizes the code, and then it uses the summary to write the python - it doesn't fully verify the source, even if I tell it to go line by line. I noticed this when a bunch of conditional checks for valid data that were in the MATLAB code weren't ported into python.

- Brent, doing something

I was very impressed with the progression of Claude's ppt skill - it has improved drastically since I used it roughly 6 months ago. Back then, the slides were very basic, and words from adjacent lines used to overlap. Now, Claude generated beautifully designed slides using the content from two web pages that I gave it the link to. I was able to fine tune the slide deck as well, by being asked how many slides I want

- Johanna, working with Claude

Testimonials

Slightly disturbing/bizarre experience: Around 4pm today ChatGPT suddenly started to use British spelling in its replies to me (like "programme" and "harmonised") for no obvious reason. So I asked why and it said "I listened to your YouTube videos".

- Paul Thompson, LinkedIn post 2026-05-05

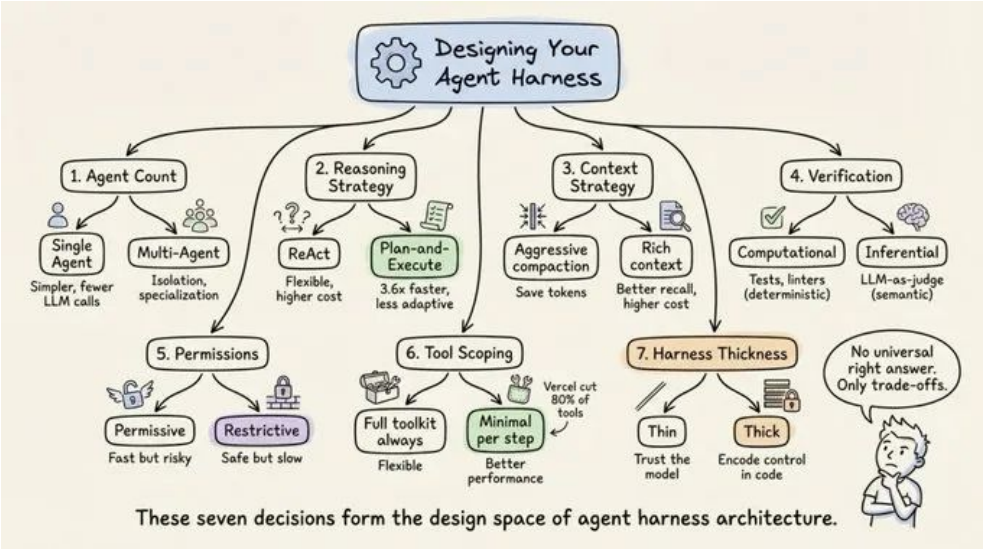
I recently reviewed a code refactoring pull request that relied heavily on an LLM. I noticed that, in splitting a Python module into two files, one of the tests was removed completely. Another thing that happened a few times is that, when copying code from one file to another, AI agents tend to remove comments from the original code.

- Michelle, maintaining a Python package

Conclusions

- “Harnessing” Agentic AI is a valuable approach to incorporating these tools into your workflow.
- Despite significant and rapid changes, there are emerging good practices for configuring and using these tools.
- They are inherently configurable and flexible by design.
    - If you are trying to configure new functionality, ask the agent harness how to do it.
- With proper caution and consideration, harnessed agentic AI can be a powerful utility to leverage in your data science workflows.

Thanks



https://blog.dailydoseofds.com/p/the-anatomy-of-an-agent-harness


Fine, I’ll do it myself.

LLM Agent as a Research Product

Paper2Agent: Reimagining Research Papers As Interactive and Reliable AI Agents

Published on Sep 8 · ★ Submitted by taesiri on Sep 9

Authors: Jiacheng Miao, Joe R. Davis, Jonathan K. Pritchard, James Zou

Abstract

Paper2Agent converts research papers into interactive AI agents to facilitate knowledge dissemination and enable complex scientific queries through natural language.

AI-generated summary

We introduce Paper2Agent, an automated framework that converts research papers into AI agents. Paper2Agent transforms research output from passive artifacts into active systems that can accelerate downstream use, adoption, and discovery. Conventional research papers require readers to invest substantial effort to understand and adapt a paper's code, data, and methods to their own work, creating barriers to dissemination and reuse. Paper2Agent addresses this challenge by automatically converting a paper into an AI agent that acts as a knowledgeable research assistant. It systematically analyzes the paper and the associated codebase using multiple agents to construct a Model Context Protocol (MCP) server, then iteratively generates and runs tests to refine and robustify the resulting MCP. These paper MCPs can then be flexibly connected to a chat agent (e.g. Claude Code) to carry out complex scientific queries through natural language while invoking tools and workflows from the original paper. We demonstrate Paper2Agent's effectiveness in creating reliable and capable paper agents through in-depth case studies. Paper2Agent created an agent that leverages AlphaGenome to interpret genomic variants and agents based on ScanPy and TISSUE to carry out single-cell and spatial transcriptomics analyses. We validate that these paper agents can reproduce the original paper's results and can correctly carry out novel user queries. By turning static papers into dynamic, interactive AI agents, Paper2Agent introduces a new paradigm for knowledge dissemination and a foundation for the collaborative ecosystem of AI co-scientists.

https://huggingface.co/papers/2509.06917

Paper2Agent

- Convert an article to an Agentic AI.
  - Create a short top level summary of findings and a chat is available to communicate details.
- Findings can be incorporated into AI tools with `MCP`.
  - Instead of citing papers, link a chatty version of the source.
- This can link code and data as well…

https://github.com/jmiao24/Paper2Agent

Paper2Agent

- Available code and workflows for a paper could be passed through `MCP` to new a new study.
- They validated this replication of work using a spatial transcriptomics analysis and dataset.

- *Instead of writing a deeply researched static summary, key findings can be reported quickly and inspected interactively through MCP.*
- *Reusable data and / or methods can shared for integration with future work.*

https://github.com/jmiao24/Paper2Agent; https://arxiv.org/pdf/2509.06917

CLAUDE.md / AGENTS.md

- This is a top level summary of what is in your code base. It is considered an easy place to add specific instructions and context in your project.
- There is currently mixed information about best practices.
  - This file is always loaded with its instructions added to context.
  - This can quickly bloat your context and weaken performance.
- There are emerging fixes, but generally keep this file to a minimum.
- The rules folder is the defaults way of dividing a large .md file into smaller contextual information.
  - The .md files in rules are dynamically loaded when the files they tagger are in context.

Agentic “Harness” - A Google AI Overview

Key Functions of an Agentic Harness

- **Secure Execution:** Instead of running locally, agents use a harness to operate within isolated sandboxes, which can monitor, allow-list commands, and manage files safely.
- **Long-Running Task Management:** Harnesses solve context fragmentation by maintaining shared state and memories across different sessions.
- **Tooling Access:** Harnesses configure tools (browsers, terminal, code editors) so agents can browse, read logs, write files, and test their own code.
- **Planning & Self-Correction:** Harnesses enable "sub-agent management" for task delegation and create self-verification loops, allowing agents to debug and fix errors autonomously.
- **Human-in-the-Loop:** Harnesses allow for, or require, human approval before executing sensitive tasks, ensuring safety in production environments.

Agentic “Harness” - A Google AI Overview

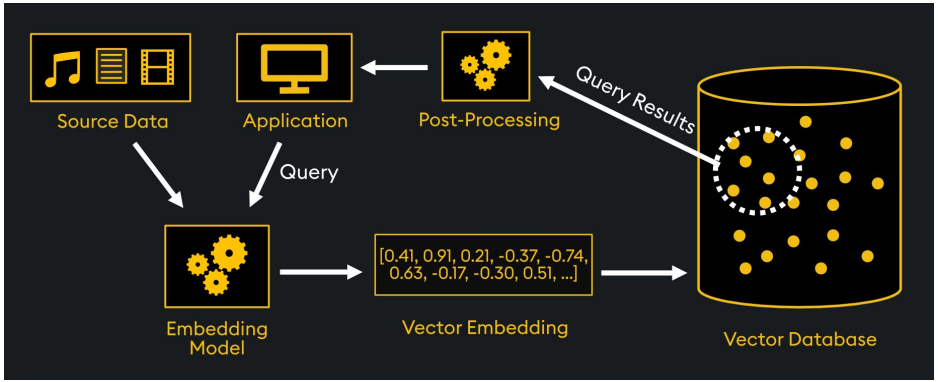
Agentic Harness Architecture
While frameworks provide tools, the harness controls the workflow:

- **Initializer Agent:** Sets up the environment in the first run.
- **Coding Agent:** Makes incremental progress across sessions.
- **Memory Compaction:** The harness summarizes previous actions to manage context token limits.
- **Fork-Join Parallelism:** Advanced harnesses can spawn sub-agents to work on different parts of a project simultaneously before merging the results.

Why Harnesses Matter (2025-2026 Shift)
As AI advances, the focus shifts from just the "engine" (the LLM model) to the "car" (the harness), allowing for reliable, multi-step agent actions often termed "vibe coding". Anthropic’s Claude Code uses this architecture to handle complex software engineering tasks.

Vector Databases

- It is a database of information that is embedded in a model's space.
- The prompt is embedded and passed to a vector database to identify relevant information based on the (semantic) distance between the prompts vector and the encoded information.
- The identified information is used to augment the prompt and improve the performance of the model.



https://kdb.ai/learning-hub/fundamentals/vector-database-101/

Model Context Protocol (MCP)

- An MCP message consists of 2 layers: the **Transport** layer and the **Data** layer.
  - The **Transport** layer defines how the MCP communicates (`STDIO` / `HTTP`).
  - The **Data** layer is what is provided to the host through the MCP.
    - **Tools**: functions the Agent can invoke for structured, programmatic responses.
    - **Resources**: data that can be added to the context of the model.
    - **Prompts**: structured queries that can be easily repeated in new contexts.
    - **Elicitations**: prompt the agent to clarify information (i.e. function inputs) from the user.
    - **Logging**: tracking requests and use to a specified output.
    - **Tasks**: asynchronous, long term interactions for task planning or pipelines.

Model Context Protocol (MCP)

- An MCP “server” can be deployed in front of a data source.
- By using this standard API, an LLM Agent can selectively access secure, contextual data for “local” use.
  - e.g. `github`
- Manage local MCPs with `docker` and `docker-desktop`!